



Next.js 6.11 — RSC Payload / React Flight

Yep 😊 This is the perfect next step because you've already built the mental model of:

Server Components → Client Components → hydration → navigation → Router Cache → rendering/caching.

Now we're going underneath that abstraction.

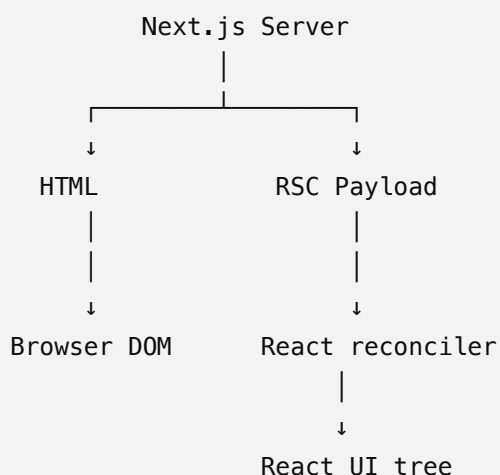
6.11 — What actually travels over the network?

When a Next.js App Router page contains Server Components, the browser doesn't simply receive:

HTML



Instead, conceptually, the server can produce **two important representations**:



Let's understand the second one.

1. What is the RSC Payload?

RSC Payload = React Server Components' serialized representation of the component tree.

It is generated by React's **Flight** protocol.

You can think of it as:

"Server, tell the browser what the Server Component tree looks like, what values it produced, and where Client Components exist."

It is **not ordinary HTML**.

For example, imagine:

```
// app/page.tsx

export default async function Page() {
  const products = await getProducts();

  return (
    <main>
      <h1>Products</h1>
      <ProductList products={products} />
    </main>
  );
}
```

And:

```
// ProductList.tsx

'use client';

export default function ProductList({ products }) {
  return (
    <div>
      {products.map(product => (
        <button key={product.id}>
          {product.name}
        </button>
      ))}
    </div>
  );
}
```

```
);  
}
```

The server executes the Server Component:

```
Page()  
  ↓  
getProducts()  
  ↓  
products  
  ↓  
<ProductList products={products} />
```



React then creates an RSC representation of this tree.

Conceptually, you can imagine something like:

```
<main>  
  <h1>Products</h1>  
  
  <ClientComponent  
    reference="ProductList"  
    props={  
      products: [...]  
    }  
  />  
</main>
```



⚠ That's **conceptual**, not the literal wire format.

The actual Flight format is an internal serialized protocol containing records/chunks.

2. Why do we need RSC Payload at all?

Here's the key question.

Suppose the server rendered:

```
<h1>Hello</h1>  
<ProductList />
```

into HTML:

```
<h1>Hello</h1>
<div>...</div>
```

The browser can display that.

But React also needs to know:

What components produced this?
Which component is a Client Component?
What props did it receive?
What should happen when navigation occurs?
What parts of the tree changed?



HTML alone doesn't provide React with all of that component-tree information.

That's where Flight comes in.

3. Think of HTML and RSC Payload as two different jobs

This is **VERY** important for interviews.

HTML

Primarily gives the browser:

"Here is the initial visual document."

```
HTML
↓
Browser
↓
Initial UI
```



RSC Payload

Gives React:

"Here is the Server Component tree/result and the information needed to construct/update the React tree."

```
RSC Payload
↓
React
↓
React Server Component tree
```



↓
UI updates

So:

HTML $\neq$ RSC Payload



They serve different purposes.

4. What happens on the initial page load?

Let's trace it.

Imagine:

GET /products



Step 1 — Server executes Server Components

Page
↓
ProductPage
↓
getProducts()
↓
ProductList



Server Components execute on the server.

Step 2 — React produces the RSC Payload

React Flight generates the serialized Server Component representation.

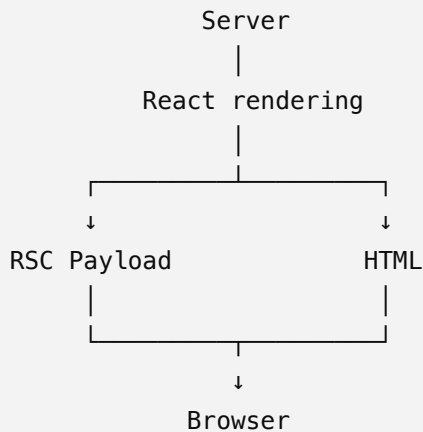
Server Component tree
↓
React Flight
↓
RSC Payload



Step 3 — Next.js uses that information to produce HTML

The result is also used to generate the HTML sent to the browser.

Conceptually:



5. Now comes the really interesting part: navigation 🚀

Suppose you're currently here:

/products

Then you click:

```
<Link href="/products/123">
  Product 123
</Link>
```

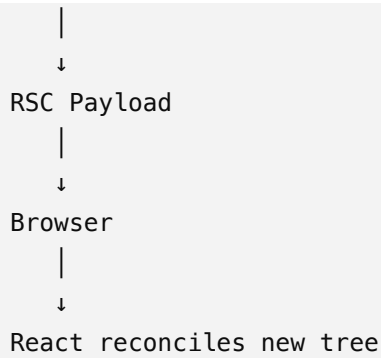
This is **not equivalent to a traditional full-page navigation**.

The browser doesn't necessarily throw away the entire document and request a completely new HTML document.

Instead, Next.js can request the **RSC representation for the new route**.

Conceptually:

```
graph TD; A[/products/] -- "click Link" --> B[/products/123/]; B -- "RSC request" --> C[Server];
```



This is one of the reasons App Router navigation can feel so fast.

6. Does HTML change during navigation?

This connects directly to your previous question. 🤖

Suppose:

Current page

/products



renders:

```
<main>
  <h1>Products</h1>
  <ProductList />
</main>
```

Then you navigate to:

/products/123



which renders:

```
<main>
  <h1>Product 123</h1>
  <ProductDetails />
</main>
```

Yes — the resulting DOM/HTML structure displayed in the browser can absolutely change.

But here's the subtle distinction:

Full document navigation

```
Browser
  ↓
GET /products/123
  ↓
new HTML document
  ↓
replace current document
```



App Router client navigation

Conceptually:

```
Browser
  ↓
request RSC payload
  ↓
React receives new server tree
  ↓
React reconciles
  ↓
DOM changes
```



So:

The DOM can change without the browser replacing the entire HTML document.

That's the distinction you were asking about earlier. 🔥

7. So does navigation fetch both HTML + RSC?

Generally, no—not in the same way as an initial document request.

For a normal App Router client-side navigation using `<Link>`:

```
Existing HTML document
  +
new RSC payload
  ↓
React updates the existing UI
```



The browser doesn't normally need a completely new HTML document for the navigation.

That's why this statement is useful:

Initial load: HTML + RSC data are involved in bootstrapping the page.

Client navigation: the RSC payload is used to update the existing React tree; the whole HTML document isn't normally re-fetched/replaced.

There are exceptions and special cases, but that's the core mental model.

8. Now: where do Client References enter?

This is where Flight gets really interesting.

Remember:

```
'use client';

export default function ProductList() {
  ...
}
```

The server cannot simply send the **JavaScript function itself** to the browser.

Functions aren't serializable like ordinary JSON data.

Instead, React needs to tell the client:

"There is a Client Component here."



Here is its reference.

Conceptually:

RSC Payload



Client Component

|

↓

Client Reference

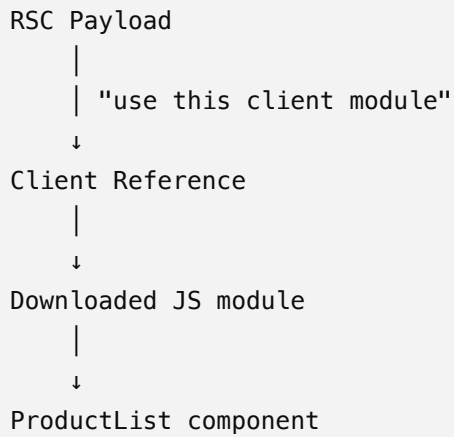
|

↓

"ProductList module #XYZ"

The browser already has the JavaScript bundle containing that Client Component.

So the payload can effectively establish the connection:



9. This explains something you've asked before

You previously asked:

"If Server Component code isn't in the browser, how does it still show on screen?"

Exactly! 😊

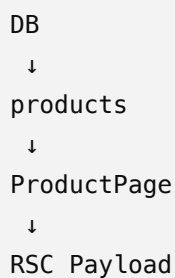
Because the browser **doesn't need the Server Component's source code**.

For example:

```
export default async function ProductPage() {  
  const products = await db.products.findMany();  
  
  return <ProductList products={products} />;  
}
```

This code stays on the server.

The server executes it:



The browser receives the **result/representation**, not the Server Component source code.

That's the fundamental RSC idea.

10. Server Reference is the opposite direction

Now we have another concept:

Server Reference

This becomes especially important with **Server Actions**.

Imagine:

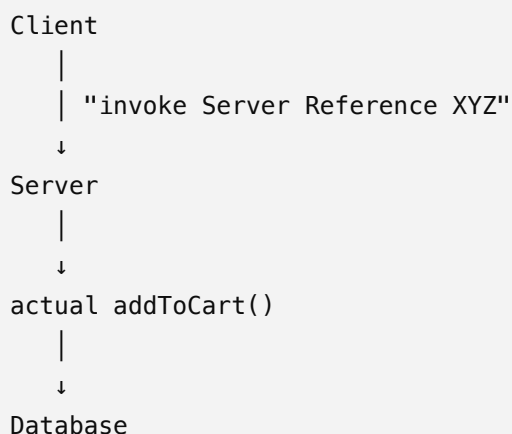
```
'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

The client doesn't receive the actual server function implementation.

Instead, React/Next.js can expose a reference to that server-side function.

Conceptually:



So we have two directions:

Client Reference

Server Component tree

↓

"Here is a Client Component"



Client Reference



Browser's JS module

Server Reference

Client



"invoke this server function"



Server Reference

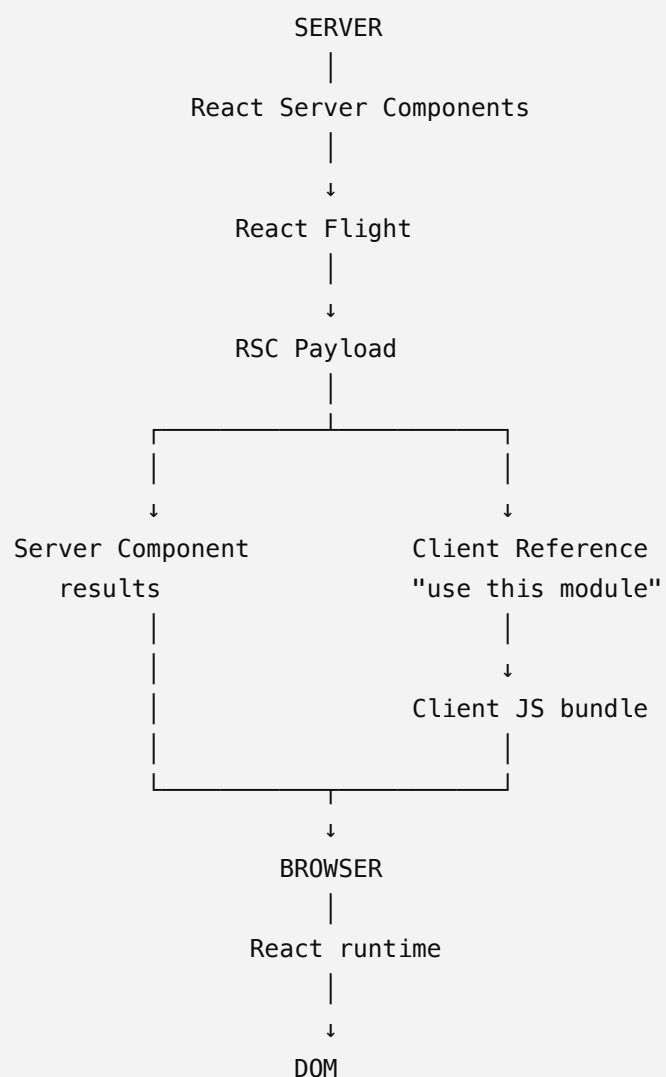


Server function

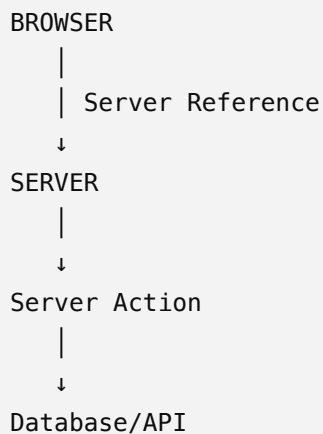


11. Put everything together 🧠

Here's the mental model I want you to remember:



And for mutations:



12. Serialization is the glue

Now we arrive at the next important concept.

The server and browser are different environments:

SERVER	BROWSER
Node.js	Browser JS
DB access	DOM
secrets	user interaction
filesystem	events

We can't just send arbitrary JavaScript objects/functions between them.

So React needs **serialization rules**.

For example, ordinary data can cross the boundary:

```
<ProductList
  products={ [
    { id: 1, name: 'Pizza' },
    { id: 2, name: 'Dosa' }
  ] }
/>
```

But this cannot simply happen:

```
<ProductList
  onClick={() => console.log('hello')}
/>
```

when crossing the Server → Client Component boundary.

Why?

Because:

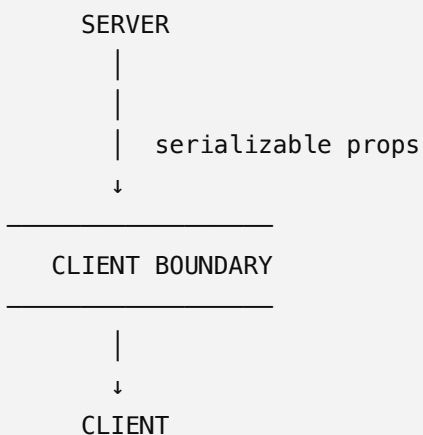
```
function
  ✗
not ordinary serializable data
```



Instead, the interactive function must exist in the Client Component's JavaScript.

13. The boundary picture

Think of this:



Server Component:

```
<ProductList products={products} />
```

can pass:

```
products
  ↓
serializable data
  ↓
Client Component
```



But:

```
<ProductList
  onSomething={serverFunction}
/>
```

isn't how ordinary Server → Client communication works.

14. One killer interview explanation 🎯

If an interviewer asks:

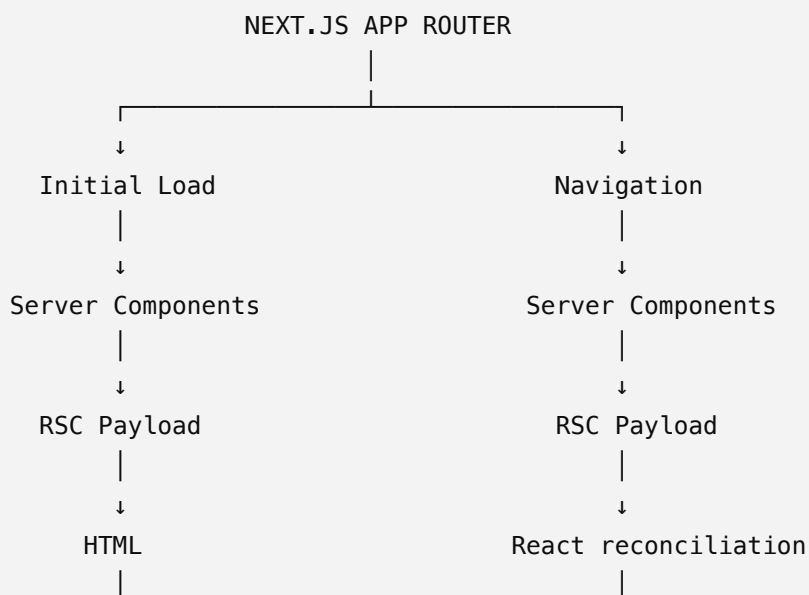
"What is the RSC payload?"

You can say:

"The RSC payload is the serialized representation produced by React Server Components using the Flight protocol. It describes the Server Component tree, the data needed by Client Components, and references to Client Components. Next.js uses it during the initial render and, importantly, during App Router client-side navigation to update the existing React tree without requiring a full document reload."

That's a **very solid mid-level answer**.

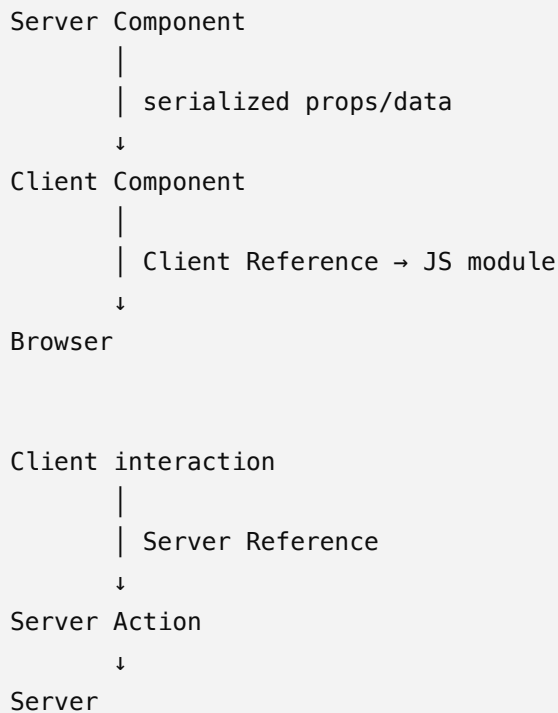
Your mental model after 6.11



↓
Browser

↓
DOM update

And the boundaries:



🔥 **Next sub-part: 6.11.1 — Flight protocol structure + actual RSC payload anatomy.**

That's where we'll look at the **weird-looking payload records/chunks**, what things like module references and serialized values represent, and then connect that directly to **Network tab → RSC requests → navigation → Router Cache**.



RSC payload-It is generated by React's **Flight** protocol.-
whast reacts flight protocol we thought nextjs server
creates them??? and also in 8th part..what do u mean by
browser already has js bundle containing client
component..usually server is initially sending client js
components right??

But:

```
<ProductList
```

Show more ▾



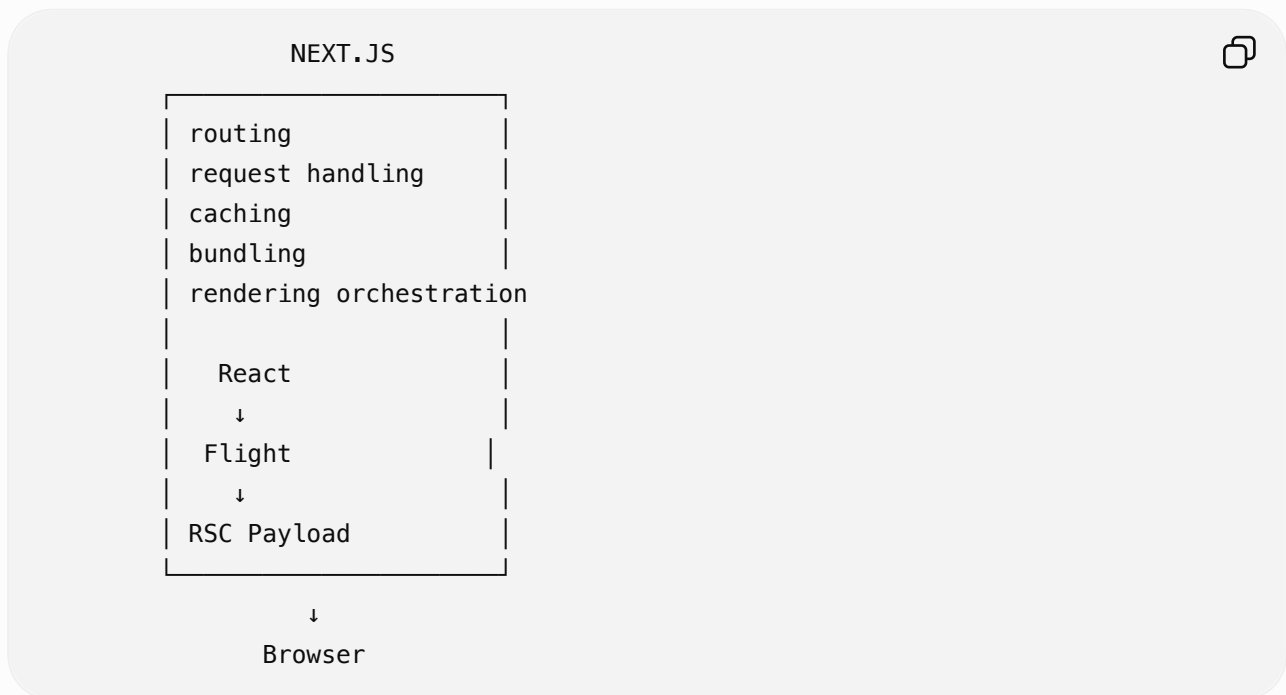
Yes — you caught **three subtle points** where my previous explanation blurred React and Next.js responsibilities. 😊 Let's clean them up carefully.

1. First: Who creates the RSC Payload — React or Next.js?

The clean answer is:

React creates the RSC/Flight representation; Next.js integrates React's Flight machinery into its server framework and transports that payload to the browser.

Think of the responsibilities like this:

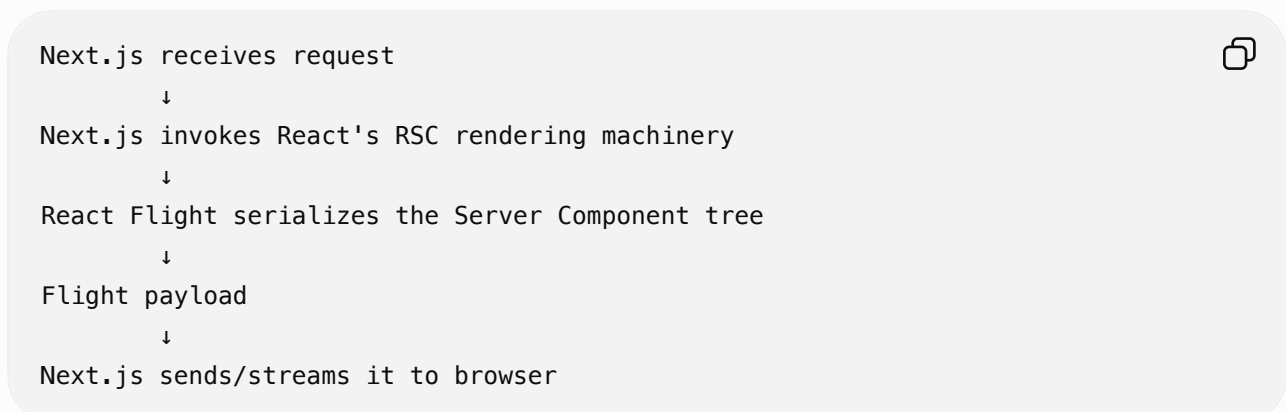


So when I said:

"RSC payload is generated by React's Flight protocol"

I didn't mean Next.js has nothing to do with it.

More precisely:



So what is React Flight?

Flight is React's protocol/mechanism for transporting the result of a React Server Component tree between server and client.

It was designed specifically to make this possible:

```
Server React tree
  ↓
serialize into Flight data
  ↓
network
  ↓
Client React
  ↓
reconstruct/interpret the tree
```



It is **not** HTTP.

It is **not** HTML.

It is **not** JSON.

It's a React-specific wire format/protocol for Server Components.

2. Why call it a "protocol"?

This is important.

When we say **protocol**, we mean:

| A defined way for two sides to communicate.

For example:

```
HTTP
Client  → Server
        GET /products
```



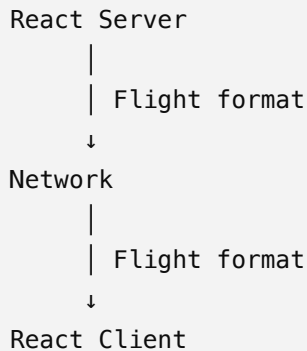
HTTP defines things like:

```
method
headers
status
body
```



Similarly, Flight defines how React communicates information about a Server Component tree.

Conceptually:



The payload contains structured records representing things such as:

- Server-rendered values
 - elements
 - strings
 - numbers
 - arrays
 - objects
- Client Component references
- module references
- etc.

The exact wire format is more complicated than this, but that's the fundamental idea.

3. "Serialized representation" — what does that actually mean?

This phrase sounds unnecessarily scary. 😂

Let's start with a normal React tree.

Suppose your Server Component conceptually produces:

```
</> TypeScript<main>  <h1>Products</h1>  <ProductList products={products} /></main>
```

React has an **in-memory representation** of this tree on the server.

Something conceptually like:

```
Element
└─ main
    ├── h1
    │   └─ "Products"
    └─ ProductList
        └─ props
            └─ products
```

But this exists inside the server's memory.

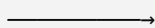
You can't send the server's JavaScript objects directly over HTTP.

So React needs to turn that information into a **network representation**.

That's serialization.

IN MEMORY

React tree
JavaScript
objects



NETWORK

Flight data
bytes/text
records

So:

Serialized representation = the Server Component tree/result converted into a format that can be transmitted over the network and interpreted by React on the other side.

That's all "serialized" means here.

4. Serialization does NOT mean "convert everything to JSON"

This is another important distinction.

You might initially think:

```
React tree
  ↓
JSON.stringify()
  ↓
JSON
```

✗ Not quite.

Flight has its **own serialization format** with special handling for React-specific things.

For example, it needs to represent:

```
Client Component references
Server references
React elements
Promises
special values
modules
etc.
```



So think:

```
Serialization
  ↓
Flight's format
```



rather than:

```
Serialization
  ↓
JSON.stringify()
```



5. Now your second question — Client JS bundle

You asked:

"browser already has JS bundle containing client component..usually server is initially sending client js components right?"

YES. 👍

My previous wording was misleading.

During the initial page load, the browser generally needs the JavaScript required for the **Client Components**.

For example:

```
</> TypeScript

'use client';
```



```
export function ProductList() {  
  ...  
}
```

This Client Component gets included in the client-side JavaScript bundle.

Conceptually:

Build time



ProductList.tsx

↓

Next.js bundling

↓

client JS bundle

↓

browser can download it

So when the RSC payload says, conceptually:

"At this location, render Client Component X"

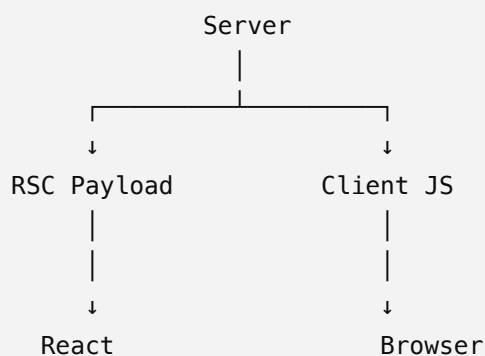


the browser needs to have access to the corresponding client module/code.

6. But there's an important nuance

The server isn't necessarily sending **the entire Client Component JS bundle as part of the RSC payload**.

These are separate things:



The RSC payload can contain a **reference to a client module**.

The JS bundle contains the **actual executable Client Component code**.

So:

```
RSC Payload
  |
  | "Client Component at this location
  |   corresponds to module X"
  ↓
Client module
  |
  ↓
actual JavaScript code
```



That's what I meant by **Client Reference**.

7. Now your really important question:

You quoted:

```
</> TypeScript

<ProductList
  onSomething={serverFunction}
/>
```



and asked:

"then how does it work?"

Excellent question.

The answer is:

There are two different situations that we must NOT mix up.

Situation A — ordinary function

Suppose:

```
</> TypeScript

function doSomething() {
  console.log("hello");
}

<ProductList onSomething={doSomething} />
```



If `ProductList` is a Client Component:

```
</> TypeScript
```



```
'use client';
```

then you **cannot** simply pass an arbitrary server-side function across the Server → Client boundary.

Why?

Because the browser cannot receive:

```
function doSomething() {  
  console.log("hello");  
}
```



as an executable server function.

The function belongs to the server environment.

8. But Server Actions are different

Now consider:

```
</> TypeScript
```



```
'use server';
```

```
export async function addToCart(productId) {  
  await db.cart.add(productId);  
}
```

This is a **Server Function / Server Action**.

And React can create a **Server Reference** for it.

Then you can do something conceptually like:

```
</> TypeScript
```



```
<ProductList action={addToCart} />
```

The important thing is:

The browser does **not** receive:

</> JavaScript



```
async function addToCart(productId) {
  await db.cart.add(productId);
}
```

Instead, the client receives a reference to the server function.

Conceptually:

SERVER



addToCart()

|

↓

Server Reference

|

↓

RSC / Flight representation

|

↓

BROWSER

Then when the client invokes it:

Browser



|

"invoke Server Reference X
with productId=123"

↓

Server

|

↓

actual addToCart()

|

↓

database

So a Server Reference is essentially an **address/handle/reference to server-side functionality**, not the function's source code.

9. This gives us a beautiful symmetry

Client Component

Server says:

"Here's a Client Component reference."



SERVER

|

Client Reference

↓

BROWSER

|

↓

Client Component code



Server Function

Client says:

"Invoke this Server Function reference."



BROWSER

|

Server Reference

↓

SERVER

|

↓

Server Function



So:

SERVER

BROWSER

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

— Client Reference —→

← Server Reference —



That's the key mental model.

10. One correction to my earlier wording

I said:

"Client Reference → JS module"

That's correct conceptually, but let's make it more precise.

A Client Reference is **not the JavaScript code itself**.

It's metadata/reference identifying a Client Component/module that the client bundler has made available.

So:

```
Client Reference
  ↓
"module X / export Y"
  ↓
client module system
  ↓
actual Client Component
```



11. And now we can define Flight properly

Here's the definition I'd like you to remember:

React Flight is the protocol/mechanism React uses to serialize and transport the Server Component tree and related references/data between the server and client.

Then:

The RSC Payload is the actual Flight-formatted data produced for a particular render/navigation.

So:

```
Flight
=
the protocol/mechanism

RSC Payload
=
the data produced using that protocol
```



Think of the analogy:

```
HTTP
  ↓
rules/protocol

HTTP response
  ↓
```



actual message

Flight



React's RSC communication protocol

RSC Payload

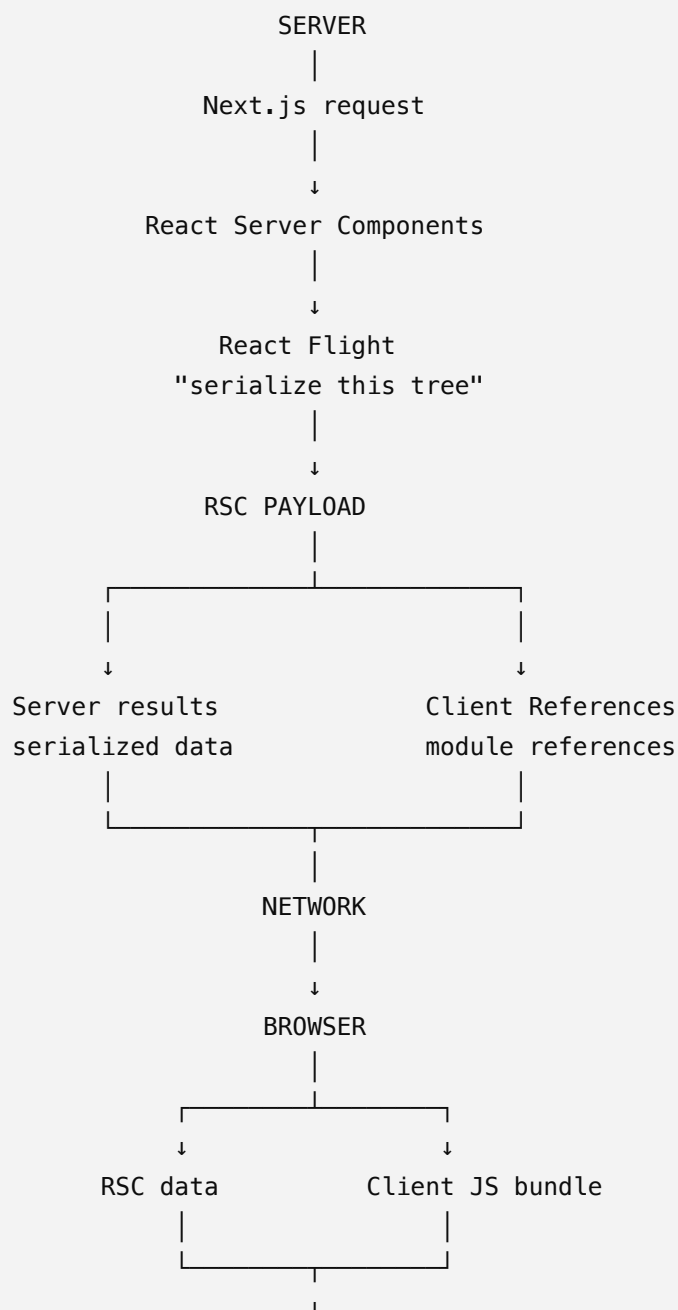


actual Flight message

That's a much cleaner mental model.

12. Final picture 🧠

Put all of 6.11 together:



React runtime



DOM

And when a Client Component needs server functionality:

Browser



Server Reference



"invoke function X"



Server



Server Action



Database/API

The three words to lock in:

Flight → *the communication protocol*

RSC Payload → *the actual serialized Flight data*

Serialization → *turning the server-side React tree/data/references into a transferable representation*

Once those three click, **RSC Payload** → **Client References** → **Server References** → **Server Actions** becomes much less mysterious. 🚀





😊 Perfect. Next we'll do **6.11.7 — the complete RSC + React Flight recap and interview questions.**

After that, **6.11.x is officially done 🏁**, and we'll start **6.12 in a new chat.**



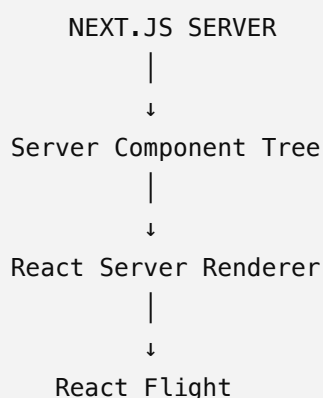
sure go ahead

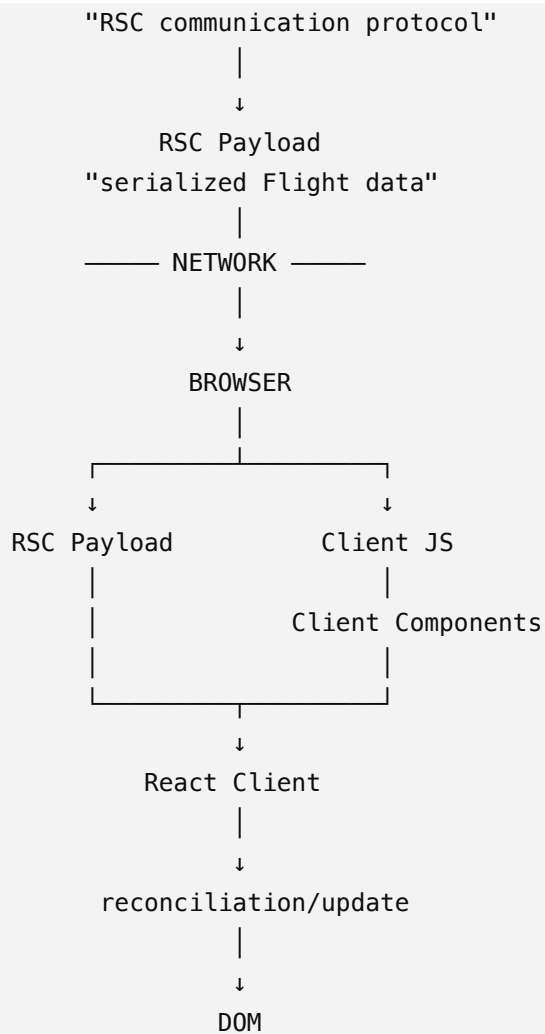
🏁 6.11.7 — Complete RSC Payload + React Flight Recap

Alright 😊 this is the **final 6.11.x lesson**. After this, we're done with RSC/Flight fundamentals and **6.12 will start in a fresh chat**, as planned.

Let's consolidate everything into one mental model first.

1. The entire RSC story in one picture





The biggest thing to remember:

Server Components don't need to be shipped to the browser as JavaScript. Their rendered result is represented in the RSC payload.

Client Components are different because their JavaScript needs to be available in the browser for interactivity.

2. Flight vs RSC Payload

This was one of your original questions.

React Flight

Think:

The protocol/mechanism used by React to transport Server Component information.

RSC Payload

Think:

The actual serialized data produced using that mechanism for a particular render/navigation.

So:

```
Flight
↓
protocol / mechanism

RSC Payload
↓
actual Flight data
```



Don't say:

"Flight is the payload."



Say:

"The RSC payload is transmitted using React Flight."



3. What does "serialized representation" mean?

Suppose the server has:

```
</> TypeScript

<Product
  name="Dosa"
  price={80}
/>
```



React has an internal representation of this component tree.

That internal representation can't simply be sent directly over HTTP.

So React converts the relevant information into a **transferable Flight representation**.

```
React tree
↓
```



```

serialization
  ↓
Flight representation
  ↓
network
  ↓
client React

```

So:

Serialized representation = a network-transferable representation of the Server Component tree, supported values, and relevant references.

And importantly:

```

Flight serialization
  ≠
JSON.stringify()

```



Flight has React-specific semantics.

4. Client References

Suppose:

```

</> TypeScript

'use client';

export default function AddButton() {
  return <button>Add</button>;
}

```



The Server Component tree can contain:

```

</> TypeScript

<AddButton />

```



The server doesn't need to send the component's source code inside the RSC payload.

Instead, Flight can represent a **Client Reference**.

Conceptually:

```

Server
|

```



```
| "Client Component X goes here"
↓
Client Reference
|
↓
Browser
|
↓
client JS module
|
↓
AddButton
```

So:

```
Client Reference
  ≠
Client Component source code
```



It's a reference to the client-side module/export.

5. Client JS vs RSC Payload

Don't mix these up.

BROWSER



```
RSC data
tree/results
references
```

```
Client JS
component code
event handlers
```

The two work together.

RSC Payload

Answers roughly:

"What Server Component result/tree should React use?"

Client JS

Provides:

"Here is the executable code for Client Components."

6. Server Functions / Server References

Now reverse the direction.

```
</> TypeScript

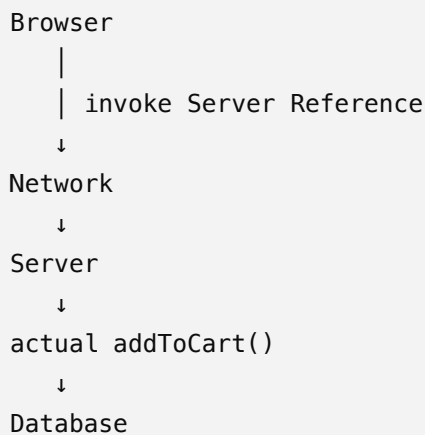
'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

The browser doesn't receive the implementation.

Instead, it can receive/use a **Server Reference**.

Conceptually:



And remember the important distinction:

function that happens to execute on server
≠
Server Function

A Server Function has explicit Server Function semantics, typically via:

```
</> TypeScript
```

```
'use server';
```

7. Ordinary function vs Server Function

Ordinary server-side function

```
</> TypeScript

function getProducts() {
  return db.products.findMany();
}
```



It may execute on the server.

But it isn't automatically remotely invocable.

```
server-side function
  ↓
runs on server
  ↓
ordinary function
```



Server Function

```
</> TypeScript

'use server';

export async function addToCart() {
  ...
}
```



```
Server Function
  ↓
Server Reference
  ↓
can be invoked through supported server-action mechanisms
```



8. Serialization boundary

Suppose:

```

</> TypeScript
export default function Page() {
  const product = {
    id: 1,
    name: "Dosa"
  };

  return <ProductCard product={product} />;
}

```

and:

```

</> TypeScript
'use client';

function ProductCard({ product }) {
  ...
}

```

The `product` data crosses:

```

Server
|
| serializable value
↓
Client boundary
|
↓
Client Component

```

But an ordinary server function cannot simply be passed the same way:

```

</> TypeScript
<ProductCard
  onSave={someServerFunction}
/>

```

unless that function has the appropriate **Server Function/reference semantics**.

9. 'use client' vs 'use server'

Keep this distinction very clear.

'use client'

```
'use client'
```

↓

Client Component boundary

↓

client module graph

↓

browser JS



```
'use server'
```

```
'use server'
```

↓

Server Function semantics

↓

Server Reference

↓

remote invocation



And:

`'use client'` does **not** simply mean "this component only renders in the browser."

That's a common oversimplification.

10. RSC navigation

Now suppose:

```
/products
```



and:

```
</> TypeScript
```

```
<Link href="/products/123">
```



is clicked.

The important flow is:

```
<Link>
```

↓

Next.js Client Router

↓

RSC request



```
↓  
Server Components execute/render  
↓  
React Flight  
↓  
RSC Payload  
↓  
Browser  
↓  
React reconciliation  
↓  
DOM update
```

Notice:

No full HTML document replacement is required for this client-side navigation.

11. DOM can change without replacing the document

This was another question you asked earlier.

Suppose:

Before

```
<main>  
  <h1>Products</h1>  
  <ProductList />  
</main>
```



After navigation

```
<main>  
  <h1>Product 123</h1>  
  <ProductDetails />  
</main>
```



The DOM can change substantially.

But:

```
DOM changed  
  ≠  
whole HTML document replaced
```



React can update the existing DOM based on the new RSC result.

That's a **very important App Router concept**.

12. Router Cache

The RSC payload received during navigation can be used by the App Router and stored in its client-side Router Cache.

Conceptually:

Server



RSC Payload



Router Cache



React



DOM



Router Cache is:

Client-side cached RSC route/segment information used by the App Router.

Don't confuse it with:

Data Cache

Full Route Cache

browser HTTP cache

JS bundle cache



They're different things.

13. Cache HIT vs MISS

HIT

Navigation



Router Cache



HIT 



reuse cached RSC information



React



DOM

MISS

Navigation



Router Cache



MISS ❌



RSC request



Server



Flight



RSC Payload



Router Cache



React



This is the core navigation-caching model.

14. Prefetching

A `<Link>` can also be prefetched in applicable situations.

Conceptually:

`<Link href="/products/123">`



prefetch



RSC Payload



Router Cache



Then:

User clicks



cached RSC information

↓

React

↓

UI

So a request can happen **before the user actually navigates**.

15. Streaming + Suspense

Now connect the last part.

Suppose:

Header → fast
Products → fast
Reviews → slow



With:

`</> TypeScript`



```
<Suspense fallback={<ReviewsSkeleton />}>
  <Reviews />
</Suspense>
```

React can conceptually produce:

Header
↓
Products
↓
ReviewsSkeleton



then later:

Reviews
↓
replace fallback



Flight allows the RSC result to be **streamed progressively**.

So:

Suspense

↓



defines boundary

Streaming



delivers results progressively

Flight

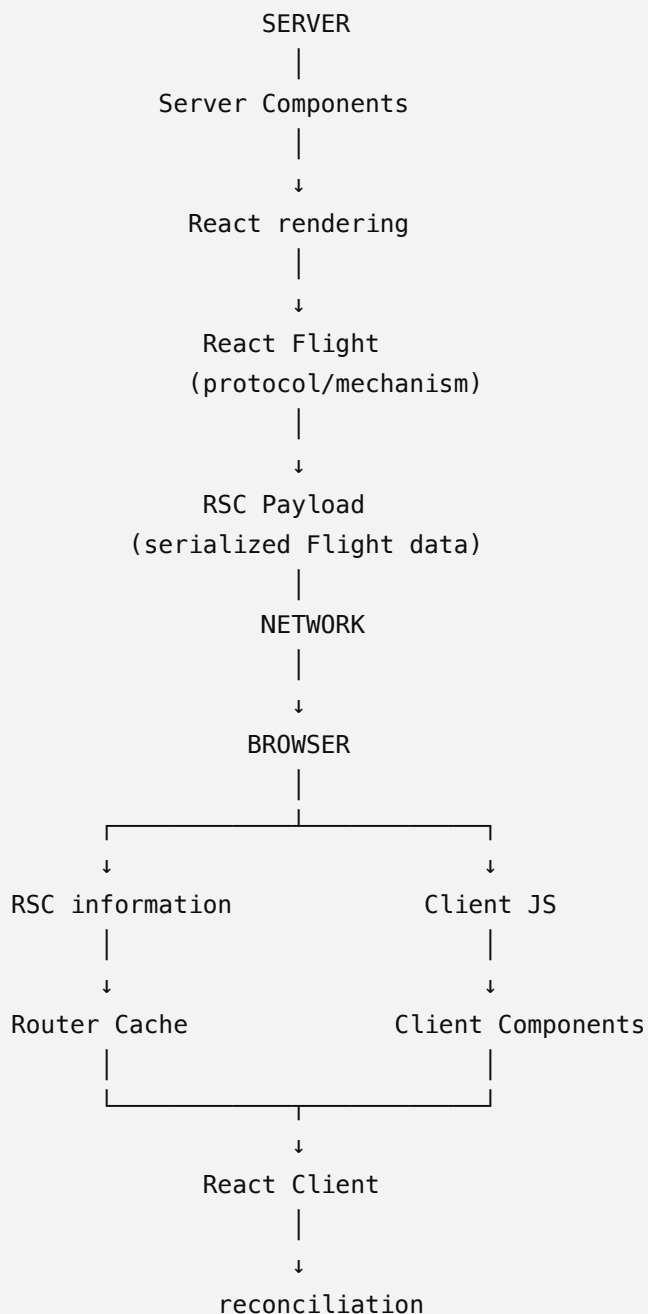


transports RSC representation

These aren't interchangeable concepts.

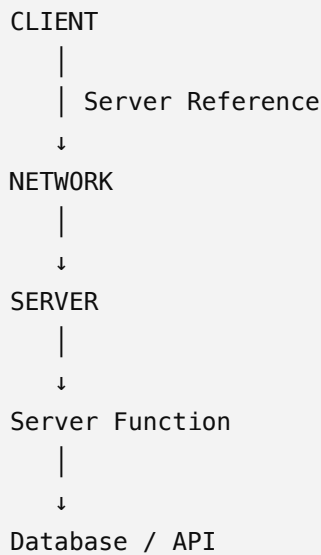
16. The complete architecture 🔥

Here's your final **6.11 mental model**:



↓
DOM

For server mutations:



🎯 17. Interview Questions

Now let's test whether you've actually got it rather than just nodding along. 😂

Q1. What is React Flight?

Answer:

React Flight is the protocol/mechanism React uses to communicate the Server Component tree between server and client.

Q2. What is the RSC Payload?

The RSC Payload is the serialized Flight data representing the Server Component result/tree, supported values, and relevant references for a particular render or navigation.

Q3. Is RSC Payload HTML?

No.

HTML

→ document/UI representation



RSC Payload

→ React Server Component representation

Q4. Is RSC Payload JSON?

Not simply.

Flight has its own serialization format and semantics beyond ordinary JSON serialization.

Q5. Does the browser receive Server Component source code?

No.

The Server Component executes on the server, and its result is represented in the RSC payload.

Q6. Does the browser receive Client Component JavaScript?

Yes.

Client Components need client-side JavaScript for their interactivity.

Q7. What is a Client Reference?

A reference in the RSC system to a Client Component/module that the browser can resolve to its client-side code.

Q8. What is a Server Reference?

A reference to a Server Function that allows the client to request its invocation without receiving the function implementation.

Q9. Can any function become a Server Function just because it runs on the server?

No.

```
runs on server
  ≠
Server Function
```



Server Function semantics are explicitly established, typically with `'use server'`.

Q10. During `<Link>` navigation, does Next.js normally download a complete new HTML document?

No.

For client-side App Router navigation, Next.js can request the RSC payload and use it to update the existing React tree and DOM.

Q11. Can the DOM structure change during client navigation?

Absolutely.

```
existing DOM
  ↓
React receives new RSC result
  ↓
reconciliation
  ↓
new DOM structure
```



But the entire browser document doesn't have to be replaced.

Q12. What does Router Cache store?

Client-side RSC route/segment information used by the App Router for navigation and reuse.

18. Your 30-second interview answer

If they ask:

"Explain React Server Components and RSC Payload in Next.js."

You can say:

"Next.js uses React Server Components to execute server-side component logic. React's Flight protocol provides the mechanism for representing and transporting the Server Component result to the client as an RSC payload. The payload contains serialized component-tree information, supported values, and references to Client Components. The client combines that information with the Client Component JavaScript and React uses it to construct or reconcile the UI. During App Router navigation, the RSC payload can be fetched and stored in the client Router Cache, allowing the existing application tree to be updated without a full document reload."

That's a very solid mid-level answer. 🎯

🏁 6.11.x — COMPLETE

You now have:

6.11 RSC Payload + React Flight



- |
- |— Flight
- |— RSC Payload
- |— serialization
- |— Client References
- |— Server References
- |— Server Functions
- |— RSC navigation
- |— Router Cache relationship
- |— prefetching
- |— streaming
- |— Suspense

🏁 DONE

And now we STOP this chat. 🙅

As planned, don't start 6.12 here.

Open a new chat and say:

"Continue Next.js 6.12"

Then we'll begin the next major piece:

6.12 — Serialization, Client References & Server References

We'll go deeper there, without mixing it back into 6.11. 😊



server function vs server side function invocation and diff

Yes — this distinction is **super important**, because "server-side function" and "Server Function" sound almost identical. 😊

1. Server-side function

A **server-side function** simply means:

A normal function whose code executes on the server.

Example:

 TypeScript 

```
async function getProducts() {  
  const products = await db.products.findMany();  
  return products;  
}  
  
export default async function Page() {  
  const products = await getProducts();  
  
  return <ProductList products={products} />;  
}
```

Here:

Page()
↓
getProducts()
↓
Database 

`getProducts()` runs on the server.

But it's just a **normal function call**:

```
Page()  
  |  
  | normal JS call  
  ↓  
getProducts()
```



There is **no network request** between **Page** and **getProducts**.

The function isn't remotely callable by the browser.

2. Server Function

A **Server Function** is different.

It is explicitly given Server Function semantics, typically with:

```
</> TypeScript  
  
'use server';  
  
export async function addToCart(productId) {  
  await db.cart.add(productId);  
}
```



Now the function can be exposed through a **Server Reference**.

Conceptually:

```
Browser  
  |  
  | invoke Server Reference  
  ↓  
NETWORK REQUEST  
  ↓  
Server  
  ↓  
addToCart(123)
```



So there's a fundamental difference:

	Server-side function	Server Function
Runs on server	✓	✓

	Server-side function	Server Function
Normal JS function	✓	Also a function, but special semantics
'use server'	✗	✓
Callable remotely from client	✗	✓ through supported mechanisms
Server Reference	✗	✓
Network involved when called from server code	✗	Not necessarily
Designed for client → server invocation	✗	✓

3. "Server-side function invocation" means something different

Suppose:

```
</> TypeScript

async function getUser() {
  return db.user.find();
}

async function Page() {
  const user = await getUser();
}
```

The invocation is:

```
Page()
|
| function call
↓
getUser()
```

Everything is happening inside the **same server execution environment**.

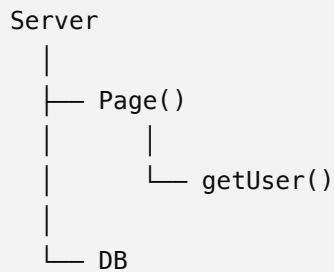
There's no:



```
graph TD; browser --> network; network --> server;
```

browser
↓
network
↓
server

It's simply:



```
graph TD; Server --> Page; Server --> DB; Page --> getUser;
```

Server
|
├─ Page()
| └─ getUser()
└─ DB

4. Server Function invocation is different

Now:



```
</> TypeScript

'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

And the browser wants to invoke it.

The browser doesn't have:

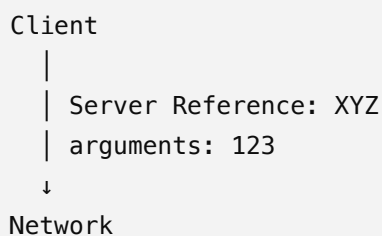


```
</> JavaScript

addToCart() // implementation
```

Instead it has a reference.

Conceptually:



```
graph TD; Client -- "Server Reference: XYZ  
arguments: 123" --> Network;
```

Client
|
| Server Reference: XYZ
| arguments: 123
↓
Network

```
↓  
↓  
Server  
|  
| resolve XYZ  
↓  
addToCart(123)
```

That's a **remote invocation**.

5. Here's the easiest way to remember it 🧠

Ask:

"Who is calling the function?"

If it's:

```
Server code  
↓  
Server function
```



that's just a normal function invocation, even if the function is a Server Function.

No special network trip is needed.

But if:

```
Browser  
↓  
Server Function
```



then the browser can't directly execute it.

It invokes the **Server Reference**, which causes communication with the server.

6. This is the subtle point

A Server Function doesn't mean:

"Every time this function is called, a network request happens."



It means:

The function has semantics that allow it to be referenced and invoked from the client.

So:

```
</> TypeScript

'use server';

export async function addToCart(id) {
  ...
}
```

If server code does:

```
</> TypeScript

await addToCart(123);
```

that's simply:

```
Server
  ↓
normal function invocation
  ↓
addToCart()
```

But when the **client** invokes it through its **Server Reference**, then:

```
Client
  ↓
Server Reference
  ↓
network
  ↓
Server Function
```

7. One more example

Normal server-side helper

```
</> TypeScript

async function calculatePrice(productId) {
  const product = await db.products.find(productId);
```

```
    return product.price * 1.18;
  }
```

Used by:

```
</> TypeScript   
  
export default async function Page() {  
  const price = await calculatePrice(123);  
  
  return <div>{price}</div>;  
}
```

Flow:

```
Page()  
  ↓  
calculatePrice()  
  ↓  
DB 
```

No client involvement.

Server Function

```
</> TypeScript   
  
'use server';  
  
export async function purchase(productId) {  
  await db.orders.create({ productId });  
}
```

Client:

```
</> TypeScript   
  
'use client';  
  
import { purchase } from './actions';  
  
export function BuyButton({ productId }) {  
  return (  
    <button onClick={() => purchase(productId)}>  
      Buy  
    </button>  
  )  
}
```

```
);  
}
```

Conceptually:

```
CLICK  
↓  
purchase(123)  
↓  
Server Reference  
↓  
NETWORK  
↓  
Server  
↓  
actual purchase(123)  
↓  
DB
```



The interview distinction

If they ask:

"What's the difference between a server-side function and a Server Function?"

Say:

"A server-side function is simply a normal function that executes in the server environment. It can be called by other server code through an ordinary JavaScript function call. A Server Function has explicit Server Function semantics, typically through `'use server'`, which allows it to be represented by a Server Reference and invoked from client code through the server-action mechanism. The function implementation remains on the server."

And the one-liner:

```
Server-side function  
= where it executes
```

```
Server Function  
= special semantics for server invocation from the client
```



That distinction is exactly what I want you to carry into 6.12.



ChatGPT can make mistakes. Check important info.